Photonics Data Engineer Intern - Technical Report

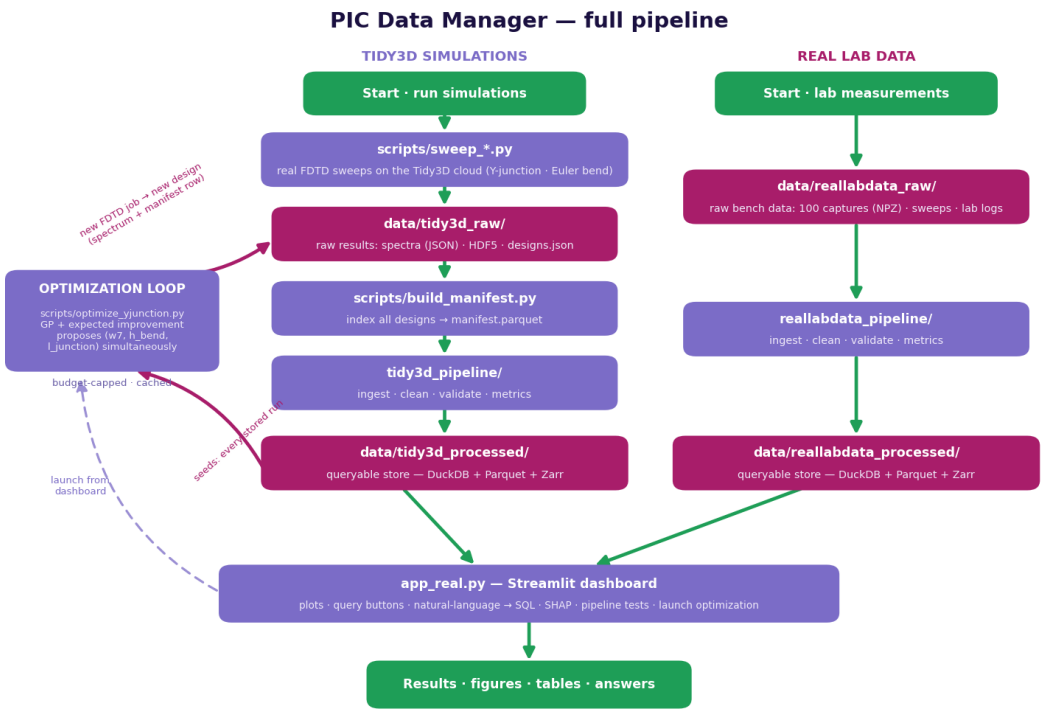
Project	Two pipelines over one storage stack: real Tidy3D FDTD simulations (tidy3d_pipeline, 53 designs) and real cascaded-MZI lab measurements (reallabdata_pipeline, 102 measurements).
Core tools	Python, pandas/NumPy/SciPy, Tidy3D (FDTD), DuckDB + Parquet + Zarr, Pydantic, Streamlit/Plotly, scikit-learn + SHAP, Claude (natural-language SQL).
Demo	streamlit run app_real.py - one command; both stores build automatically on first launch.
Headline results	Best Y-junction 0.155 dB (-0.54 dB via the optimization loop); Euler bend best 0.005 dB. The lab MZM is a two-stage cascade: the multiplicative two-stage fit (R^2 = 0.99997) gives per-stage V_pi 3.20 / 4.60 V; the composite response has -3 dB bandwidth 0.319 MHz and extinction 16.7 dB.
Validation	17 pytest tests + 11 live dashboard checks: known-answer analysis tests, ingest round-trips + idempotency, metadata-vs-manifest validation, raw-file -> stored-metric end-to-end verification, SHA-256 artifact integrity.

1. Architecture

Each dataset has its own pipeline package, but both use the same structure. The data model is a set of Pydantic entities, and each entity gets its own table in DuckDB: Component, SimulationRun, RawArtifact, AnalysisResult and OptimizationRecord for the simulations; Device, Measurement, RawArtifact, AnalysisResult and LabLog for the lab bench. DuckDB holds the metadata and metrics and answers all SQL queries. The measurement payloads live next to it as files: Parquet for data that is tabular - rows x named columns, like a transmission spectrum or a Bode table - and Zarr for data that is N-dimensional arrays, like the 25 x 4096 waveform cubes and the $|E|^2$ field map, where chunking lets one trace be read without loading the whole cube.

Layer	tidy3d_pipeline (simulations)	reallabdata_pipeline (lab bench)
Data model (Pydantic), one DuckDB table each	Component, SimulationRun, RawArtifact, AnalysisResult, OptimizationRecord	Device, Measurement, RawArtifact, AnalysisResult, LabLog
Ingestion adapter	ingest.py - manifest + JSON spectra + HDF5 field maps	ingest.py - NPZ sweeps/captures, device card + lab logs (Markdown)
Store / query	store.py - DuckDB + Parquet + Zarr	store.py - DuckDB + Parquet + Zarr
Analysis	insertion loss, 3 dB bandwidth (computed at ingest for every spectrum)	fringe (incl. two-stage fit), Bode, step features; predictive model - cross-validated RandomForest swing prediction with residual anomaly flags and SHAP attribution
Entry points	cli.py (summary / query / filter) + optimize.py (stretch)	cli.py (metrics / captures / features / logs)

2. Pipeline flow chart



3. Schema

Entity (one DuckDB table each)	Key fields
Component	id, type, name, function, library/source, geometry (swept + fixed parameters), material/platform, expected performance metrics
SimulationRun	run_id, component_id, tool + version, wavelength range, boundary conditions, mesh summary, per-run geometry snapshot, settings, timestamp
RawArtifact	repo-relative path, format, SHA-256 checksum, size
AnalysisResult	metric name / value / units / definition, analysis code version, timestamp
OptimizationRecord	iteration, parameters, objective, best-so-far

Format	Holds	Why
DuckDB	metadata, provenance, metrics - the SQL engine (incl. JSON geometry extraction); can also query the Parquet files in place	one analytical engine for catalog + payloads
Parquet	tabular payloads: transmission spectra, sweep/Bode tables, the design manifest	columnar, compressed, pandas/DuckDB-native
Zarr	N-D arrays: $\|E\|^2$ field map (475x308), 100 waveform cubes (25 traces x 4096 samples)	chunked - slice one trace without reading the cube; attrs carry acquisition metadata

4. Ingestion flow

The adapter is the single seam between a raw result and its stored metrics. Per file:

- **Read** (tidy3d_pipeline/ingest.py) reads the manifest row + JSON spectrum per design, and raw HDF5 when present; reallabdata_pipeline/ingest.py reads the NPZ static sweep, FRA, 100 captures, and the Markdown device card / lab logs.
- **Clean** (ingest.py, _spectrum_df) normalizes every spectrum to a common (wavelength_um, transmission) frame.
- **Validate** (tidy3d_pipeline/analysis.py, check_spectrum) applies physical sanity checks: monotonic wavelength axis, T in [0, 1.05], no NaNs; bad spectra are rejected, not stored.
- **Store** (tidy3d_pipeline/store.py, reallabdata_pipeline/store.py) writes provenance rows to DuckDB and payloads to Parquet/Zarr behind a checksummed RawArtifact with a repo-relative path.
- **Analyze** (analysis.py in each package) computes the metrics into analysis_results with no manual step.
- **Query** (store.py methods; cli.py; app_real.py) exposes runs for a component, compare variants, filter by geometry, and retrieve metrics in the package API, the CLI, and the dashboard, where each query button shows its exact SQL beside a natural-language query box (Claude writes read-only SQL).

The original Tidy3D Jupyter notebooks that were run can be found under /notebooks.

A suite of tests validates ingestion and analysis, which can be found in the dashboard under "Pipeline Tests".

5. Design tradeoffs

- **DuckDB + Parquet + Zarr over SQLite** - one analytical engine over catalog and payloads; payloads routed by shape (tables -> Parquet, arrays -> Zarr).
- **Free-form JSON geometry** - new component types need no schema migration; validation lives in Pydantic and in metadata-vs-manifest tests.
- **Content-hashed IDs** - idempotent ingestion over append-only history.
- **RandomForest + SHAP over a neural net** - 22-100 samples per dataset; small-sample-honest cross-validation with exact TreeSHAP attribution.
- **Raw data committed, stores regenerated** - both stores rebuild automatically on first launch; artifact paths are repo-relative so committed DuckDB files resolve on any checkout.

6. Stretch: component optimization

scripts/optimize_yjunction.py closes the loop against real FDTD (see the loop in the flow chart). A Gaussian-process surrogate is first fitted to all 30 existing sweep designs (free seed data). Expected improvement then proposes new (w7, h_bend, l_junction) combinations, varying all three parameters simultaneously; each proposal runs a real Tidy3D job and is stored as an OptimizationRecord. The loop is launchable from the dashboard's Optimization tab, which shows the GP loss landscape, per-iteration progress for new evaluations only, and the improvement relative to the best pre-existing design.

Result of a real 12-evaluation run: the GP's second proposal (w7 = 1.43 um, h_bend = 2.49 um, l_junction = 2.10 um) measured **0.149 dB**, beating the best sweep design (0.174 dB) by 0.025 dB and the original reference design (0.155 dB).

7. Assumptions, limitations, and production metadata (simulation integration)

Assumptions:

- The simulation's transmission T is a fraction of the input power, so insertion loss can be computed as -10 log10(T).
- The timestamp on each record is when the data was ingested into the store, not when the simulation actually ran on the cloud.

Limitations:

- The sweeps are small: 31 Y-junction and 22 Euler-bend designs over only a few values per parameter, so conclusions rest on few data points.
- When ingesting from the manifest, the Tidy3D software version and the cloud task ID are not recorded, so a run cannot be matched to the exact solver version or cloud job that produced it.

- Zarr files (waveforms, field maps) have no checksums yet, so unlike the Parquet files their integrity cannot be re-verified later.

Metadata a production version would add:

- Which software versions produced each run: the Tidy3D version and a list of every installed package, so a result can be recreated later even after software updates change behavior.
- Solver logs, true execution time, and the cost of each run.

8. Next steps

1. **Cloud storage** - move the Parquet and Zarr files off the local computer into a cloud storage bucket (e.g. Amazon S3) so the store is not tied to one machine.

2. **CI/CD** - run the test suites automatically on every code change, comparing the computed metrics against a saved "golden file" so any changed number fails the build instead of going unnoticed.

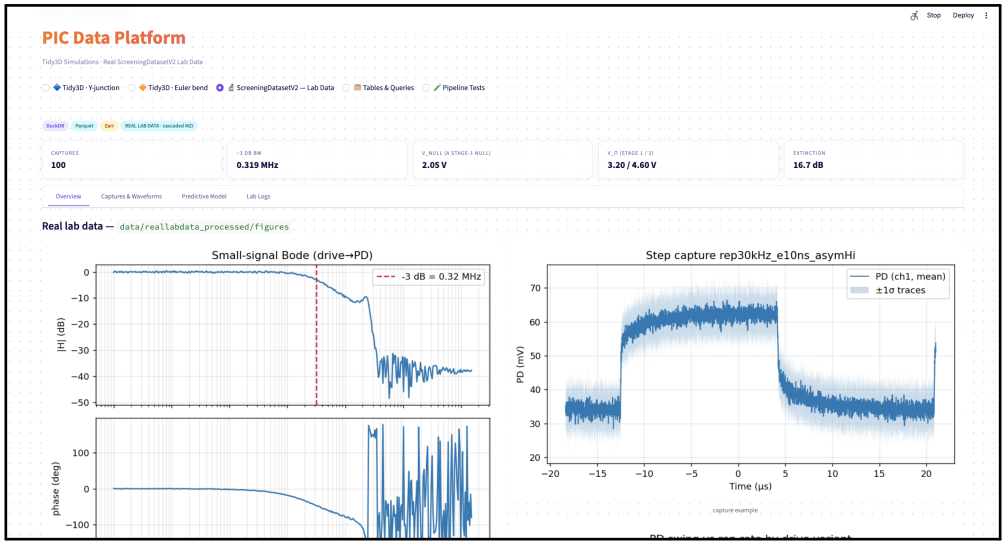
3. **Richer run records** - record which software versions produced each run, plus the solver's log, the true execution time, and the cost of each run.

4. **Speed up the optimization loop** - it currently runs one FDTD job at a time, so a study is slow. Simulating several jobs in parallel would cut the time.

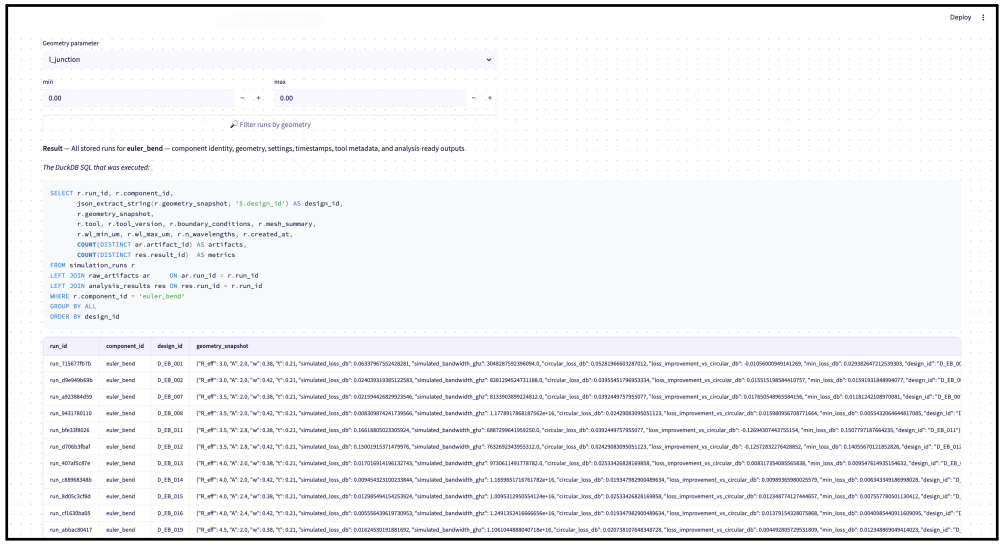
9. Dashboard



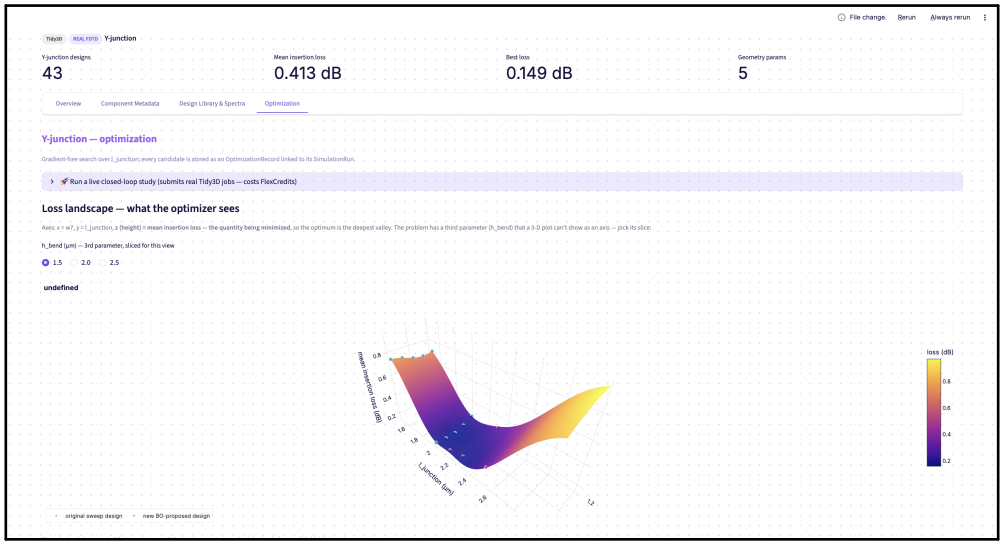
Tidy3D component view



Lab-data overview



Tables & Queries console



Optimization loop